

M07 - PostgreSQL + Data Modeling

13-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Build governed PostgreSQL schemas and analytical models with keys, history and validation.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M02-M03 SQL and M05 Python-for-DE foundations. You should understand joins, keys, constraints, transactions, and validation.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE07-01 - Verify PostgreSQL connection and database/schema context	5
DE07-02 - Tables, data types and explicit DDL	7
DE07-03 - Primary keys, foreign keys and constraints	9
DE07-04 - Transactions, DML and UPSERT	11
DE07-05 - Indexes and EXPLAIN evidence	14
DE07-06 - Operational vs analytical modeling and normalization	17
DE07-07 - Facts, dimensions and star-schema grain	19
DE07-08 - Surrogate keys and referential integrity	21
DE07-09 - Slowly Changing Dimension Type 2 history	23
DE07-10 - Build and validate a small analytical database	26
DE07-11 - Bulk loading with COPY / staging boundaries	28
DE07-12 - Schemas, roles and least-privilege boundaries	30
DE07-13 - Schema change, backup/restore and handoff validation	32

Prerequisite, tool and outcome map

ASSUMED

Complete M02-M03 SQL and M05 Python-for-DE foundations. You should understand joins, keys, constraints, transactions, and validation.

MODULE PURPOSE

Build governed PostgreSQL schemas and analytical models with keys, history and validation.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE07-01 - Verify PostgreSQL connection and database/schema context

1. Why it matters

Prove the active database, role and schema search context before changing any object. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Prove the active database, role and schema search context before changing any object.

3. Prerequisite / what you already know

Complete M02-M03 SQL and M05 Python-for-DE foundations. You should understand joins, keys, constraints, transactions, and validation.

4. Picture it in your head

Before changing data, know the exact server -> database -> schema -> table chain. Picture it as an address; a correct SQL statement at the wrong address is still a failure.

5. New words before we use them

A PostgreSQL server (cluster) can contain several databases. A client connection works inside one database at a time, and that database contains schemas. Unqualified table names are resolved through `search_path`. This is why a correct statement against the wrong database or schema is still a serious mistake. M07 reuses the PostgreSQL installation introduced earlier; do not reinstall simply because this module begins.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL Tutorial for Beginners

Assigned scope: 0:00–17:12

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=SpflwAYaKk)

URL: <https://www.youtube.com/watch?v=SpflwAYaKk>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT current_database() AS db, current_user AS db_user, current_schema() AS current_schema;
SHOW search_path;
SELECT version();
```

Connect to the dedicated stage07_de database. Run the context queries, then create no objects until the database/user/search path evidence is saved.

8. Expected result / what you should see

`current_database()` is stage07_de; `current_user` is known; `search_path` is recorded; no destructive statement ran before context proof.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Verify PostgreSQL connection and database/schema context: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actual. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If context is wrong, stop. Reconnect to stage07_de rather than editing setup SQL to fit the wrong database.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Before changing data, know the exact server -> database -> schema -> table chain. 3. Run the earliest independent control: For Verify PostgreSQL connection and database/schema context: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Close video/book. Reconnect in a fresh query tab or psql session and prove context again. Explain why the GUI tree selection alone is insufficient evidence.

13. Validate your result

For Verify PostgreSQL connection and database/schema context: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Verify PostgreSQL connection and database/schema context in your own words. Start with the purpose: Prove the active database, role and schema search context before changing any object. Then name one realistic failure mode and explain how you would prove the result using this check: For Verify PostgreSQL connection and database/schema context: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actual.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Verify PostgreSQL connection and database/schema context with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-02.

DE07-02 - Tables, data types and explicit DDL

1. Why it matters

Choose explicit PostgreSQL types/defaults from business meaning and create reproducible DDL. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Choose explicit PostgreSQL types/defaults from business meaning and create reproducible DDL.

3. Prerequisite / what you already know

DE07-01 mastered.

4. Picture it in your head

DDL is the blueprint for the table: columns, types, defaults and constraints define what data is allowed before any row is loaded.

5. New words before we use them

Types are part of the data contract. Use date for calendar dates, timestampz when an instant/time-zone meaning matters, integer/bigint for whole numbers, numeric(p,s) when exact decimal precision matters, boolean for true/false, and text for variable text. DEFAULT supplies a value when a column is omitted; it does not magically replace an explicitly supplied NULL. DDL should live in scripts so a second learner can reproduce the schema.

6. Useful video / source

Source: freeCodeCamp — detailed PostgreSQL course

Assigned scope: 41:37–55:55

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=qw--VYLpxG4)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

CREATE TABLE source.suppliers (supplier_id text PRIMARY KEY, supplier_name text NOT NULL, country text NOT NULL, active boolean NOT NULL DEFAULT true, rating numeric(3,2) CHECK (rating BETWEEN 0 AND 5));
Create source.suppliers from the course specification, then inspect information_schema.columns instead of trusting the GUI alone.

8. Expected result / what you should see

Column definitions match the written contract; dates/numbers are not stored as text merely to avoid parsing; DDL is saved.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Tables, data types and explicit DDL: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If a value does not fit the type, decide whether the data is invalid or the type contract was wrong. Do not weaken the type automatically.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: DDL is the blueprint for the table: columns, types, defaults and constraints define what data is allowed before any row is loaded. 3. Run the earliest independent control: For Tables, data types and explicit DDL: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create source.carriers with a text business key, required name, active boolean default, joined_on date and exact rating/score field. Justify each type in comments.

13. Validate your result

For Tables, data types and explicit DDL: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Tables, data types and explicit DDL in your own words. Start with the purpose: Choose explicit PostgreSQL types/defaults from business meaning and create reproducible DDL. Then name one realistic failure mode and explain how you would prove the result using this check: For Tables, data types and explicit DDL: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Tables, data types and explicit DDL with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-03.

DE07-03 - Primary keys, foreign keys and constraints

1. Why it matters

Use database constraints to reject invalid identity, relationships, missing required values and impossible ranges. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Use database constraints to reject invalid identity, relationships, missing required values and impossible ranges.

3. Prerequisite / what you already know

DE07-02 mastered.

4. Picture it in your head

Imagine every real entity carrying an ID badge. A key should identify the intended record, duplicates are two badges claiming the same identity, and NULL means the value is unknown/not supplied rather than zero.

5. New words before we use them

PRIMARY KEY identifies a row and is unique/non-null. UNIQUE protects alternate keys. FOREIGN KEY requires the referenced parent key to exist. NOT NULL makes presence mandatory. CHECK enforces a row-level predicate. Constraint failures during a controlled test are good evidence: they prove the database boundary is working. Never remove a valid integrity rule simply to make deliberately bad practice data load.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL constraints

Assigned scope: 2:29:24–2:49:15 + 3:16:41–3:25:04

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=qw--VYLpxG4)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

INSERT INTO source.suppliers VALUES ('S001','Demo','IN',true,4.50); -- deliberate failures, one at a time: -- duplicate S001; NULL supplier_name; rating 6.00 Run one valid supplier insert, then one duplicate-key, one NULL-required-value and one out-of-range test. Save each error and map it to the constraint.

8. Expected result / what you should see

Each invalid state is rejected for the expected reason; valid rows remain; no constraint was disabled/deleted as a shortcut.

9. Why it worked

This works because use database constraints to reject invalid identity, relationships, missing required values and impossible ranges is made explicit before the result is accepted. Your independent check is: For Primary keys, foreign keys and constraints: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

Read the exact violated constraint and identify the business rule. Repair the bad input or model—not the evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Imagine every real entity carrying an ID badge. 3. Run the earliest independent control: For Primary keys, foreign keys and constraints: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design a parent carrier table plus a child route table. Prove one orphan route is rejected by the FK and roll back/clean only the test state.

13. Validate your result

For Primary keys, foreign keys and constraints: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Primary keys, foreign keys and constraints in your own words. Start with the purpose: Use database constraints to reject invalid identity, relationships, missing required values and impossible ranges. Then name one realistic failure mode and explain how you would prove the result using this check: For Primary keys, foreign keys and constraints: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Primary keys, foreign keys and constraints with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-04.

DE07-04 - Transactions, DML and UPSERT

1. Why it matters

Group related writes in transactions, prove rollback/commit behavior and use ON CONFLICT only with an explicit same-key business rule. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Group related writes in transactions, prove rollback/commit behavior and use ON CONFLICT only with an explicit same-key business rule.

3. Prerequisite / what you already know

DE07-03 mastered.

4. Picture it in your head

A transaction groups related inserts/updates/deletes into one atomic unit. UPSERT adds “insert if new, update if existing” but still needs a trustworthy conflict key.

5. New words before we use them

BEGIN starts an explicit transaction, COMMIT makes its changes durable, and ROLLBACK discards uncommitted changes. The strongest rollback proof records before, inside-transaction and after-rollback states. UPSERT is not “update anything that conflicts”; ON CONFLICT must target a meaningful unique key and define which attributes are safe to update. Transactions preserve atomicity boundaries but cannot make incorrect business logic correct.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL DML/transactions

Assigned scope: 2:54:45–3:16:41

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=qw--VYLpxG4)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
BEGIN;
UPDATE source.customers SET region='
South'
WHERE customer_id='
C001'
;
SELECT customer_id,region FROM source.customers WHERE customer_id='
C001'
;
ROLLBACK;
SELECT customer_id,region FROM source.customers WHERE customer_id='
C001'
;
```

Run the rollback demonstration, then perform one controlled UPSERT on an existing supplier or dimension row.

8. Expected result / what you should see

Before/inside/after evidence proves rollback; post-rollback source state matches baseline; UPSERT target is a real unique key and only intended fields update.

9. Why it worked

The operation is trustworthy because related changes cross the commit boundary together; a before/after state check proves partial updates are not being mistaken for success. Your independent check is: For Transactions, DML and UPSERT: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If a GUI autocommits unexpectedly, stop and reproduce in one explicit transaction block. A COMMIT already issued cannot be undone by ROLLBACK.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A transaction groups related inserts/updates/deletes into one atomic unit. 3. Run the earliest independent control: For Transactions, DML and UPSERT: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Change C002 inside a new transaction, prove temporary state, roll back, then write an UPSERT for a different key with a comment explaining conflict policy.

13. Validate your result

For Transactions, DML and UPSERT: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case.

14. Teach it back in your own words

Without notes, explain Transactions, DML and UPSERT in your own words. Start with the purpose: Group related writes in transactions, prove rollback/commit behavior and use ON CONFLICT only with an explicit same-key business rule. Then name one realistic failure mode and explain how you would prove the result using this check: For Transactions, DML and UPSERT: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Transactions, DML and UPSERT with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-05.

DE07-05 - Indexes and EXPLAIN evidence

1. Why it matters

Propose an index from a query pattern and interpret EXPLAIN evidence without demanding an Index Scan on tiny data. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Propose an index from a query pattern and interpret EXPLAIN evidence without demanding an Index Scan on tiny data.

3. Prerequisite / what you already know

DE07-04 mastered.

4. Picture it in your head

Use a lab notebook mental model: first attempt, observed result, repair, and changed retry are separate entries. If you erase the failed attempt, you lose the evidence of learning.

5. New words before we use them

An index is a separate structure maintained as table rows change, so it trades storage/write work for potential read efficiency. Index the access pattern, not every column. EXPLAIN shows the planner estimate; EXPLAIN ANALYZE executes and adds actual evidence, so use it carefully. On tiny course tables a sequential scan may remain cheaper even after a sensible index exists. The lab is about evidence, not forcing a plan shape.

6. Useful video / source

Source: PostgresConf — Basic understanding of EXPLAIN ANALYZE

Assigned scope: Beginner EXPLAIN/ANALYZE class; use the index-scan / sequential-scan sections

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=Kdjz2e8HYPU)

URL: <https://www.youtube.com/watch?v=Kdjz2e8HYPU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
EXPLAIN SELECT * FROM source.orders WHERE customer_id='C003'
```

```
;
```

```
CREATE INDEX IF NOT EXISTS idx_orders_customer_id ON source.orders(customer_id);
```

```
EXPLAIN SELECT * FROM source.orders WHERE customer_id='C003'
```

```
;
```

Capture before/after plans for the customer lookup and write only what actually changed. Record that Seq Scan on tiny data is acceptable.

8. Expected result / what you should see

Claim is supported by observed plan evidence; no “index always faster” statement; index matches a real predicate/join/order pattern.

9. Why it worked

The reasoning separates correctness from performance: first prove the result, then inspect the planner/execution evidence to see whether the engine can avoid unnecessary work. Your independent check is: For Indexes and EXPLAIN evidence: record the actual before/after state and verify the lesson requirement with an independent control. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If the planner still uses Seq Scan, do not force an index. Explain table size/selectivity and keep the evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Use a lab notebook mental model: first attempt, observed result, repair, and changed retry are separate entries. 3. Run the earliest independent control: For Indexes and EXPLAIN evidence: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Choose a different repeated filter/join from the lab, propose one index, capture before/after EXPLAIN and state one write/storage cost.

13. Validate your result

For Indexes and EXPLAIN evidence: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Claim is supported by observed plan evidence; no "index always faster" statement; index matches a real predicate/join/order pattern.

14. Teach it back in your own words

Without notes, explain Indexes and EXPLAIN evidence in your own words. Start with the purpose: Propose an index from a query pattern and interpret EXPLAIN evidence without demanding an Index Scan on tiny data. Then name one realistic failure mode and explain how you would prove the result using this check: For Indexes and EXPLAIN evidence: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Indexes and EXPLAIN evidence with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-06.

DE07-06 - Operational vs analytical modeling and normalization

1. Why it matters

Explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation.

3. Prerequisite / what you already know

DE07-05 mastered.

4. Picture it in your head

Operational models optimize safe updates of current entities; analytical models optimize historical measurement and slicing. Picture two different floor plans for different workloads.

5. New words before we use them

Operational models usually separate business entities so one fact or attribute has an authoritative home and updates do not require changing dozens of copied values. Analytical models often reorganize data around measurable events and descriptive dimensions. Normalization is not a moral score: the right boundary follows workload, ownership, integrity and grain. Before deciding table boundaries, state what one row means and which business key identifies it.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling

Assigned scope: Main modeling class — operational vs analytical thinking + dimensional-model context

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=uF76d5EmdtU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Operational: customers -> orders -> order_items. Customer name/region live once in customers. Analytical: fact_sales repeats di Inspect source.customers/orders/order_items and list three update/anomaly problems that would appear if customer_name and region were copied into every order line.

8. Expected result / what you should see

You distinguish harmful duplicated attributes from intentional repeated foreign keys at fact grain; every proposed table has a written grain.

9. Why it worked

This works because explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation is made explicit before the result is accepted. Your independent check is: For Operational vs analytical modeling and normalization: record the actual before/after state and verify the lesson requirement with an independent control. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If you are arguing “normalized good / denormalized bad,” return to workload + integrity + grain and explain the trade-off.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Operational models optimize safe updates of current entities; analytical models optimize historical measurement and slicing. 3. Run the earliest independent control: For Operational vs analytical modeling and normalization: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Take a wide carrier-delivery row and sketch an operational decomposition. Then state which repeated keys would be acceptable in an analytical fact.

13. Validate your result

For Operational vs analytical modeling and normalization: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: You distinguish harmful duplicated attributes from intentional repeated foreign keys at fact grain; every proposed table has a written grain.

14. Teach it back in your own words

Without notes, explain Operational vs analytical modeling and normalization in your own words. Start with the purpose: Explain why operational models often reduce update anomalies while analytical models reshape data for understandable aggregation. Then name one realistic failure mode and explain how you would prove the result using this check: For Operational vs analytical modeling and normalization: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Operational vs analytical modeling and normalization with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-07.

DE07-07 - Facts, dimensions and star-schema grain

1. Why it matters

Declare fact grain first, identify dimensions/measures, build a simple star and reconcile grouped totals. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Declare fact grain first, identify dimensions/measures, build a simple star and reconcile grouped totals.

3. Prerequisite / what you already know

DE07-06 mastered.

4. Picture it in your head

Think of grain as camera zoom: “one row per order” and “one row per order item” are different zoom levels. A calculation is trustworthy only when you know which level the row represents.

5. New words before we use them

A fact table records measurable events at a declared grain and stores dimension foreign keys. Dimensions hold descriptive context used to group/filter. In this lab warehouse.fact_sales is one row per COMPLETED order line—not one row per order. One order can legitimately have several fact rows. Aggregating by region changes result grain to one row per region, but the regional values must reconcile to the overall fact measure.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling

Assigned scope: Main modeling class — facts, dimensions, grain and star-schema reasoning

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=uF76d5EmdtU)

URL: <https://www.youtube.com/watch?v=uF76d5EmdtU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT c.region, SUM(f.gross_sales) AS revenue FROM warehouse.fact_sales f JOIN warehouse.dim_customer c ON c.customer_key=f.customer_key GROUP BY c.region ORDER BY c.region;
```

Write the fact grain sentence before querying. Produce revenue by region and product category; reconcile both grouped totals to overall fact revenue.

8. Expected result / what you should see

fact_sales grain is correct; 12 rows, qty 30, revenue 14130.00; region/category grouped results each sum back to 14130.00.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Facts, dimensions and star-schema grain: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If totals inflate, inspect join cardinality and measure grain before using DISTINCT. DISTINCT is not a repair for unexplained fan-out.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Think of grain as camera zoom: “one row per order” and “one row per order item” are different zoom levels. 3. Run the earliest independent control: For Facts, dimensions and star-schema grain: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design a delivery fact at one row per delivered parcel line with date/customer/product-like dimensions. Identify additive measures and one measure that could be duplicated at the wrong grain.

13. Validate your result

For Facts, dimensions and star-schema grain: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Facts, dimensions and star-schema grain in your own words. Start with the purpose: Declare fact grain first, identify dimensions/measures, build a simple star and reconcile grouped totals. Then name one realistic failure mode and explain how you would prove the result using this check: For Facts, dimensions and star-schema grain: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Facts, dimensions and star-schema grain with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-08.

DE07-08 - Surrogate keys and referential integrity

1. Why it matters

Separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references.

3. Prerequisite / what you already know

DE07-07 mastered.

4. Picture it in your head

A surrogate key is a warehouse-controlled identity tag. Foreign keys connect facts/history to the correct dimension version and should never be guessed from descriptive text.

5. New words before we use them

A business key such as `customer_id` comes from the source/business. A surrogate key is generated inside the warehouse and has no business meaning by itself. The dimension should keep the business key for traceability while facts use the surrogate key to reference the exact warehouse row. This becomes especially valuable when one business key can have multiple historical dimension versions. Referential integrity must be proven, not assumed.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling

Assigned scope: Main modeling class — dimensional model keys/grain; Teaching Book adds surrogate-key rules

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=uF76d5EmdtU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT customer_key,customer_id,customer_name,region FROM warehouse.dim_customer ORDER BY customer_key;
```

-- orphan checks should return 0 Display business/surrogate key pairs, then run explicit orphan checks for `customer_key` and `product_key`.

8. Expected result / what you should see

orphan customer/product refs = 0; invalid FK insert is rejected; natural key remains visible in dimension; surrogate key is not treated as user-facing meaning.

9. Why it worked

This works because separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references is made explicit before the result is accepted. Your independent check is: For Surrogate keys and referential integrity: record the actual before/after state and verify the lesson requirement with an independent control. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If an orphan exists, trace source key -> dimension lookup/load -> fact load. Never replace unknown keys with arbitrary existing surrogate keys.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A surrogate key is a warehouse-controlled identity tag. 3. Run the earliest independent control: For Surrogate keys and referential integrity: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Inside a transaction, attempt a fact row with a nonexistent product_key, save the FK error, then roll back. Explain why storing product_name as the relationship key would be weaker.

13. Validate your result

For Surrogate keys and referential integrity: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: orphan customer/product refs = 0; invalid FK insert is rejected; natural key remains visible in dimension; surrogate key is not treated as user-facing meaning.

14. Teach it back in your own words

Without notes, explain Surrogate keys and referential integrity in your own words. Start with the purpose: Separate business/natural identifiers from warehouse surrogate keys and prove zero orphan fact references. Then name one realistic failure mode and explain how you would prove the result using this check: For Surrogate keys and referential integrity: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Surrogate keys and referential integrity with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-09.

DE07-09 - Slowly Changing Dimension Type 2 history

1. Why it matters

Preserve attribute history with effective dates/current flags instead of overwriting the past. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Preserve attribute history with effective dates/current flags instead of overwriting the past.

3. Prerequisite / what you already know

DE07-08 mastered.

4. Picture it in your head

A pipe connects one command's output to the next command's input; redirection sends output to a file. Picture commands as small boxes connected by arrows.

5. New words before we use them

SCD Type 2 retains multiple versions of a dimension business key. A current row is closed and a new row is inserted when a tracked attribute changes. Typical fields are `effective_from`, `effective_to` and `is_current`. A transaction keeps the close-old/open-new steps together. The toy lab deliberately teaches the pattern but does not pretend one CHECK constraint solves all production overlap/concurrency cases.

6. Useful video / source

Source: TechLake — Slowly Changing Dimensions and Types

Assigned scope: Whole focused SCD class; concentrate on Type 1 vs Type 2 history

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=-XjPvozI-So)

URL: <https://www.youtube.com/watch?v=-XjPvozI-So>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

-- C003 moves North ->

East effective 2026-09-01 BEGIN;

UPDATE warehouse.dim_customer_scd2 SET effective_to='2026-08-31'

,is_current=false WHERE customer_id='C003'

AND is_current=true;

-- insert new current C003 East version COMMIT;

Run the SCD2 lab. Prove C003 has two versions and exactly one `is_current=true`, then narrate what history would be lost by an in-place UPDATE.

8. Expected result / what you should see

Old version remains; new version starts on intended date; exactly one current version for changed key; effective dates do not overlap in the taught example.

9. Why it worked

This works because preserve attribute history with effective dates/current flags instead of overwriting the past is made explicit before the result is accepted. Your independent check is: For Slowly Changing Dimension Type 2 history: record the actual before/after state and verify the lesson requirement with an independent control. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If two current rows remain, stop and repair the close-old/insert-new transaction. Do not delete the old history row.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A pipe connects one command's output to the next command's input; redirection sends output to a file. 3. Run the earliest independent control: For Slowly Changing Dimension Type 2 history: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Apply a changed future segment update to a different customer in a fresh SCD2 copy. Save before/after history and check exactly one current row.

13. Validate your result

For Slowly Changing Dimension Type 2 history: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Old version remains; new version starts on intended date; exactly one current version for changed key; effective dates do not overlap in the taught example.

14. Teach it back in your own words

Without notes, explain Slowly Changing Dimension Type 2 history in your own words. Start with the purpose: Preserve attribute history with effective dates/current flags instead of overwriting the past. Then name one realistic failure mode and explain how you would prove the result using this check: For Slowly Changing Dimension Type 2 history: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Slowly Changing Dimension Type 2 history with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-10.

DE07-10 - Build and validate a small analytical database

1. Why it matters

Build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation.

3. Prerequisite / what you already know

DE07-01 through DE07-09 mastered.

4. Picture it in your head

Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence.

5. New words before we use them

A successful CREATE/INSERT run is not enough. A reliable build has objective invariants: declared grain, row counts, eligibility/exclusion rules, unique business grain, zero orphans and monetary/quantity totals that reconcile. The setup is intentionally rerunnable at the module level; validation must be rerun after any material model change.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to data modeling

Assigned scope: Revisit only the model-design parts needed before building the analytical database

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=uF76d5EmdtU)

URL: <https://www.youtube.com/watch?v=uF76d5EmdtU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT COUNT(*) AS fact_rows, SUM(qty) AS total_qty, SUM(gross_sales) AS total_revenue FROM
warehouse.fact_sales;
```

-- Expected baseline: 12, 30, 14130.00 Run the canonical setup/build then final validation. Explain every control: 12 fact rows, qty 30, revenue 14130.00, cancelled O1005 contributes zero, no duplicate (order_id,line_no), zero orphans.

8. Expected result / what you should see

All baseline controls pass and region totals East 3710, North 2390, South 1750, West 6280 sum to 14130.

9. Why it worked

This works because build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation is made explicit before the result is accepted. Your independent check is: For Build and validate a small analytical database: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If one control fails, isolate eligibility -> dimension lookup -> fact grain -> measure calculation. Do not adjust expected totals to match a broken load.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence. 3. Run the earliest independent control: For Build and validate a small analytical database: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

From a clean stage07_de rebuild, execute the build and validation without looking at the teacher SQL except when repairing. Save a validation transcript.

13. Validate your result

For Build and validate a small analytical database: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Build and validate a small analytical database in your own words. Start with the purpose: Build the supplied operational + star schemas and prove row count, grain, referential integrity and revenue reconciliation. Then name one realistic failure mode and explain how you would prove the result using this check: For Build and validate a small analytical database: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Build and validate a small analytical database with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-11.

DE07-11 - Bulk loading with COPY / staging boundaries

1. Why it matters

Load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy.

3. Prerequisite / what you already know

DE07-10 mastered.

4. Picture it in your head

Imagine a ZIP as a sealed travel bag. Extracting opens a working copy; copying keeps the original in place; moving changes location; renaming changes the label, not the contents.

5. New words before we use them

COPY is a PostgreSQL SQL command; when it names a file, the server process reads/writes from the server environment and is constrained by server-side permissions. psql \copy is a psql meta-command: the client reads/writes the local file and streams data through the connection. In a learner workstation this distinction explains why a local path can work with \copy but fail with server-side COPY. Staging is a receiving area: load permissively enough to inspect, reconcile source business rows, then transform into final constrained tables.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use COPY / loading section if present; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=qw--VYLpxG4)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
CREATE TABLE source.stg_customers(customer_id text,customer_name text,region text,segment text);
```

```
-- psql client-side: -- \copy source.stg_customers FROM 'customers_stage.csv'
```

```
WITH (FORMAT csv, HEADER true);
```

```
SELECT COUNT(*) FROM source.stg_customers;
```

Load the bundled customers_stage.csv into source.stg_customers using psql \copy or documented pgAdmin Import/Export. Save source business-row count and staging COUNT(*).

8. Expected result / what you should see

Source and staging business-row counts reconcile; header is not counted; learner explains COPY vs \copy and staging vs final tables.

9. Why it worked

This works because load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy is made explicit before the result is accepted. Your independent check is: For Bulk loading with COPY / staging boundaries: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If a local path fails with COPY, do not change file permissions blindly. First determine whether the command is server-side COPY or client-side \copy.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Imagine a ZIP as a sealed travel bag. 3. Run the earliest independent control: For Bulk loading with COPY / staging boundaries: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Load carriers_stage.csv to a new staging table, reconcile rows, flag one malformed/unknown region row before any final constrained insert.

13. Validate your result

For Bulk loading with COPY / staging boundaries: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

14. Teach it back in your own words

Without notes, explain Bulk loading with COPY / staging boundaries in your own words. Start with the purpose: Load CSV into a staging boundary and distinguish server-side COPY from psql client-side \copy. Then name one realistic failure mode and explain how you would prove the result using this check: For Bulk loading with COPY / staging boundaries: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Bulk loading with COPY / staging boundaries with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-12.

DE07-12 - Schemas, roles and least-privilege boundaries

1. Why it matters

Design schema/role boundaries that grant only the operations a workload needs. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Design schema/role boundaries that grant only the operations a workload needs.

3. Prerequisite / what you already know

DE07-11 mastered.

4. Picture it in your head

Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. Always know which room and dialect your query is using.

5. New words before we use them

PostgreSQL roles can be login identities or group-like privilege containers. Database CONNECT, schema USAGE/CREATE and table SELECT/INSERT/UPDATE/DELETE are different powers. Object ownership is stronger than ordinary grants. A reporting workload normally should not run as the database owner or superuser. Least privilege means writing down what the workload needs and explicitly what it does not need. Default privileges/future objects are a separate consideration from existing tables.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use roles/privileges section if present; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
CREATE ROLE stage07_reader NOLOGIN;
```

```
GRANT USAGE ON SCHEMA warehouse TO stage07_reader;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA warehouse TO stage07_reader;
```

Design warehouse_reader and source_loader privilege matrices. If your local account is allowed, create only a NOLOGIN reader and inspect grants;

otherwise do the design evidence without changing system privileges.

8. Expected result / what you should see

Reader has no write/drop/ownership requirement; loader does not receive warehouse-owner powers; design distinguishes schema USAGE from table privileges.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Schemas, roles and least-privilege boundaries: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If permission errors appear, inspect `current_user`, ownership and actual grants. Do not switch the application to `postgres/superuser` as the repair.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. 3. Run the earliest independent control: For Schemas, roles and least-privilege boundaries: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

For a fresh reporting service, state exact database/schema/table privileges, owner boundary and how future-table access would be handled. Reject `GRANT ALL` as an unexplained shortcut.

13. Validate your result

For Schemas, roles and least-privilege boundaries: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Schemas, roles and least-privilege boundaries in your own words. Start with the purpose: Design schema/role boundaries that grant only the operations a workload needs. Then name one realistic failure mode and explain how you would prove the result using this check: For Schemas, roles and least-privilege boundaries: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Schemas, roles and least-privilege boundaries with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE07-13.

DE07-13 - Schema change, backup/restore and handoff validation

1. Why it matters

Make a reproducible schema change, create a logical backup, restore to a separate target and validate the restored copy. This matters because database modeling and constraint mistakes become persistent shared state, so later queries cannot safely repair the ambiguity.

2. What you are learning

Make a reproducible schema change, create a logical backup, restore to a separate target and validate the restored copy.

3. Prerequisite / what you already know

DE07-12 mastered.

4. Picture it in your head

Treat schema change like a controlled migration: record before-state, apply versioned DDL, verify data/contracts, and prove you can restore or rebuild from documented artifacts.

5. New words before we use them

Schema change should be scripted, reviewed and validated rather than hidden in GUI clicks. `pg_dump` creates a logical export; custom format (-Fc) is restored with `pg_restore` and supports selective restore features. A backup file is not recovery proof until a restore to a separate database/environment succeeds and the same controls pass. Never place a real password in the command text or evidence. A handoff must include build/migration scripts, grains, control totals, role assumptions, backup metadata, restore validation and known limitations.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use backup/restore/schema-change support only; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=qw--VYLpxG4)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

`ALTER TABLE source.products ADD COLUMN IF NOT EXISTS active boolean NOT NULL DEFAULT true;` -- terminal examples: -- `pg_dump -Fc -d stage07_de -f stage07_de.backup -- createdb stage07_restore_test -- pg_restore -d stage07_restore_test stage07_de.backup` Apply the active-column migration and prove its definition. If `pg_dump/pg_restore` are available, dump and restore to `stage07_restore_test`, then rerun final controls.

8. Expected result / what you should see

Migration is scripted; backup metadata recorded; restore target is separate; post-restore controls match baseline where runtime is available. If runtime is unavailable, limitation is recorded—not guessed.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Schema change, backup/restore and handoff validation: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually en. The proof is enforced database state, keys/constraints and reconciled data, not DDL text alone.

10. Common beginner mistakes

If restore fails, keep original database untouched, inspect dump format/version/target/permissions, and retry only against the separate test target.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Treat schema change like a controlled migration: record before-state, apply versioned DDL, verify data/contracts, and prove you can restore or rebuild from documented artifacts. 3. Run the earliest independent control: For Schema change, backup/restore and handoff validation: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a handoff checklist and a changed reversible migration for a new nullable source attribute. Include rollback/recovery thought, validation and restore target separation.

13. Validate your result

For Schema change, backup/restore and handoff validation: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Schema change, backup/restore and handoff validation in your own words. Start with the purpose: Make a reproducible schema change, create a logical backup, restore to a separate target and validate the restored copy. Then name one realistic failure mode and explain how you would prove the result using this check: For Schema change, backup/restore and handoff validation: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually en.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Schema change, backup/restore and handoff validation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.